

LESSON 8

ROS2 Concepts Rapid Review

You used ROS2 + MoveIt2 in your thesis. Refresh, don't relearn.

- 8.1 Core Concepts Table
- 8.2 Topics vs Services vs Actions
- 8.3 Minimal Publisher/Subscriber
- 8.4 Key Commands
- 8.5 YOUR Thesis Pipeline
- 8.6 Underwater Robotics Context

8.1 Core Concepts

Concept	What It Is	Analogy
Node	A process doing one job	Worker in factory
Topic	Named channel for streaming data	Radio frequency
Publisher	Sends messages to topic	Radio broadcaster
Subscriber	Listens to a topic	Radio listener
Service	Request-reply (synchronous)	Calling customer support
Action	Long task with progress feedback	Food delivery with tracking
Launch File	Starts multiple nodes	Startup script
Parameter	Node configuration value	App settings
tf2	Coordinate transforms between frames	GPS for robot parts
Bag	Records all topic data	Flight recorder

8.2 Topics vs Services vs Actions

TOPIC (Publisher/Subscriber):

Direction: One-way, continuous stream

Example: `/camera/image` -> publishes frames at 30Hz

Use when: Sensor data, state updates, anything continuous

Pattern: Fire and forget. No response expected.

SERVICE (Client/Server):

Direction: Two-way, one-time request-reply

Example: `/get_map` -> client asks, server responds with map

Use when: Quick queries, configuration changes

Pattern: Blocks until response received.

ACTION (Client/Server + Feedback):

Direction: Two-way with progress updates

Example: `/navigate_to_goal` -> "50% there... 80%... done!"

Use when: Long-running tasks (navigation, manipulation)

Pattern: Non-blocking. Can be cancelled mid-way.

8.3 Minimal Publisher & Subscriber

```

import rclpy
from rclpy.node import Node
from std_msgs.msg import String

# ■■■ PUBLISHER ■■■
class MyPublisher(Node):
    def __init__(self):
        super().__init__("my_publisher")
        self.pub = self.create_publisher(String, "my_topic", 10)
        self.timer = self.create_timer(1.0, self.callback)

    def callback(self):
        msg = String()
        msg.data = "Hello ROS2!"
        self.pub.publish(msg)
        self.get_logger().info(f"Published: {msg.data}")

# ■■■ SUBSCRIBER ■■■
class MySubscriber(Node):
    def __init__(self):
        super().__init__("my_subscriber")
        self.sub = self.create_subscription(
            String, "my_topic", self.callback, 10)

    def callback(self, msg):
        self.get_logger().info(f"Received: {msg.data}")

def main():
    rclpy.init()
    node = MyPublisher() # or MySubscriber()
    rclpy.spin(node)
    rclpy.shutdown()

```

8.4 Key ROS2 Commands

```

ros2 topic list           # list all active topics
ros2 topic echo /my_topic # see messages on a topic
ros2 topic info /my_topic # see type, publishers, subs
ros2 topic hz /my_topic  # see publish rate

ros2 node list            # list all running nodes
ros2 node info /my_node   # see node details

ros2 service list         # list all services
ros2 service call /srv ... # call a service

ros2 launch pkg launch.py # run a launch file
ros2 bag record -a         # record all topics
ros2 bag play recording.db3 # replay recorded data

ros2 param list           # list all parameters
ros2 param get /node param # get parameter value

```

8.5 YOUR Thesis Pipeline (Know This Cold!)

!! YOUR PIPELINE: Camera -> Point Cloud -> GraspGen -> MoveIt2 -> UR10

1. Intel RealSense D435 captures RGB-D images
2. Depth data -> 3D point cloud of the scene
3. GraspGen (diffusion model) generates 6-DOF grasp poses
4. Grasp pose transformed: camera frame -> robot base frame (tf2!)
5. MoveIt2 plans collision-free trajectory
6. UR10 robot arm executes the grasp

ROS2 concepts you USED:

- Topics: camera images, point clouds, grasp poses streaming
- tf2: camera_frame -> base_link coordinate transform
- MoveIt2: motion planning (this is an Action internally)
- Launch files: starting entire perception-to-action pipeline
- Parameters: camera settings, planning parameters

BE READY TO EXPLAIN:

- Why ROS2 over ROS1? (DDS middleware, real-time, lifecycle)
- How tf2 transforms work (parent-child frames)
- What MoveIt2 does (motion planning, collision checking)
- How you handled sim-to-real transfer

8.6 Underwater Robotics Context (Andromeda)

UNDERWATER CHALLENGES:

- No GPS underwater (signals don't penetrate water)
- No WiFi/radio (use acoustic communication - slow, lossy)
- Limited visibility (turbid water, no sunlight at depth)
- Pressure increases ~1 atm per 10m depth
- Corrosion from saltwater

SENSOR SUITE (what Andromeda likely uses):

DVL (Doppler Velocity Log):

- Measures velocity relative to seabed using sound
- Primary sensor for underwater dead reckoning

IMU (Inertial Measurement Unit):

- Acceleration + angular velocity
- Provides attitude (pitch, roll, yaw)

Depth Sensor:

- Pressure-based depth measurement
- Very accurate (pressure is reliable)

Sonar:

- Forward-looking: obstacle detection
- Side-scan: seabed mapping
- Multibeam: 3D bathymetry
- Like underwater LiDAR but using sound

Camera:

- Works in clear water, close range
- Used for inspection, object detection

LOCALIZATION APPROACH:

Dead reckoning: DVL + IMU integration (drifts over time)

Acoustic positioning: USBL/LBL beacon systems

Surface GPS fix: when vehicle surfaces

Sensor fusion: combine all for best estimate

COMMUNICATION:

Underwater: acoustic modem (low bandwidth, high latency)

Surface: WiFi/radio/satellite

Tethered (ROV): fiber optic cable (high bandwidth)